Relazione Tecnica: Piattaforma Provinciale di Monitoraggio degli Sfiori

Franco Visibile

Indice

1 Introduzione **2**

2 Architettura dei Dati (Backend) **2**
2.1 Il Modello Profilo 2

3 Logica di Business e Autenticazione **2**
3.1 Il Workflow di Login 2
3.2 Registrazione Utente 4
3.3 Dashboard di Amministrazione 4

4 Interfaccia Utente (Frontend) **4**
4.1 Struttura Base e Ereditarietà 5
4.2 Dashboard Differenziate 5

5 Sicurezza **6**

6 Conclusioni **6**

7 Architettura del Modulo Operativo e Gestione Dati **6**
7.1 Modellazione ER e Relazioni One-to-One 6
7.2 Gestione Transazionale dei Form 7

8 Data Engineering e Interoperabilità (Excel) **7**
8.1 Algoritmo di Importazione (ETL) 7
8.2 Esportazione e Denormalizzazione 8

9 Mappa e Ottimizzazione delle Query **8**
9.1 Soluzione al problema N+1 8

10 Gestione Documentale Sicura **9**

11 Interfaccia Utente (Frontend Logic) **9**

12 Ingegneria del Frontend e User Experience (UX) **9**
12.1 Pattern di Visualizzazione Dati (Dashboard) 10
12.2 Gestione dell'Input Complesso (Accordion Pattern) 10
12.3 Integrazione Mappa (Leaflet.js) 11
12.3.1 Iniezione dei Dati (Data Injection) 11
12.3.2 Gestione delle Anomalie 11
12.4 Gestione Documentale e Upload 11

13 Conclusioni Generali **12**

1 Introduzione

Il presente documento descrive l'architettura tecnica del modulo di accesso e gestione profili del *Sistema Provinciale di Monitoraggio Sfiori*. L'obiettivo di questo modulo è garantire un accesso sicuro alla piattaforma, discriminando le funzionalità disponibili in base all'ente di appartenenza dell'utente (AATO o AqA). Il sistema è sviluppato utilizzando il framework **Django** e sfrutta un'architettura MVT (Model-View-Template).

2 Architettura dei Dati (Backend)

La gestione degli utenti si basa sull'estensione del modello standard di Django. Per soddisfare i requisiti specifici del dominio, è stato implementato un pattern *One-to-One* per associare metadati aggiuntivi a ciascun utente.

2.1 Il Modello Profilo

Nel file `models.py`, è definita la classe `Profilo`. Questa classe funge da estensione del modello `User` nativo di Django. La caratteristica fondamentale è il campo `ente`, che determina i permessi e la dashboard di destinazione.

```python
class Profilo(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    ente = models.CharField(max_length=10, choices=ENTI)

    def __str__(self):
        return f"{self.user.username} - {self.ente}"
```

Listing 1: Definizione del modello Profilo

Le scelte per il campo ente sono vincolate a due valori costanti:

- **AATO**: Ufficio d'Ambito (Ente di controllo).
- **AqA**: Gestore del servizio idrico.

3 Logica di Business e Autenticazione

La logica applicativa è contenuta principalmente in `views.py`. Il sistema non utilizza un login generico, ma implementa un *Dispatcher* personalizzato che indirizza l'utente verso aree riservate distinte.

3.1 Il Workflow di Login

La funzione `login_view` gestisce l'autenticazione. Il flusso logico è il seguente:

1. Il sistema riceve le credenziali (username e password) via POST.
2. Viene invocata la funzione `authenticate()` per verificare la validità delle credenziali.
3. Se l'utente è valido, viene recuperato il `Profilo` associato.
4. In base al valore del campo `ente`, avviene il reindirizzamento:
    - Se `ente == 'AATO'` → redirect a `'aato'`.
    - Se `ente == 'TEA'` → redirect a `'tea'`.

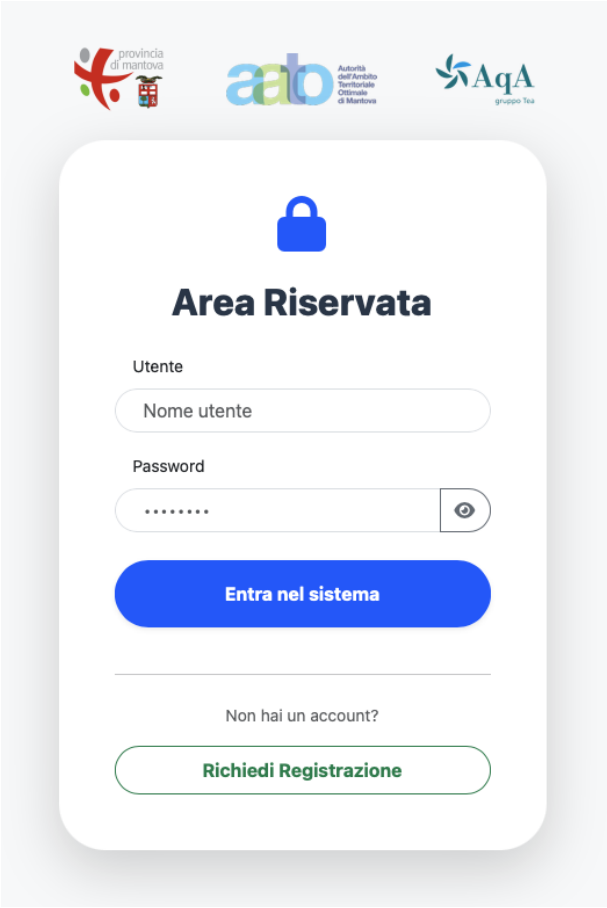

Figura 1: Processo di autenticazione.

```
if user:
    login(request, user)
    ente = user.profilo.ente
    if ente == 'AATO':
        return redirect('aato')
    elif ente == 'TEA':
        return redirect('tea')
```

Listing 2: Logica di Dispatching in login_view

3.2 Registrazione Utente

La vista `registra_utente` utilizza un `Form` personalizzato (`UserProfiloForm`) definito in `forms.py`. Questa vista gestisce una transazione atomica implicita che crea due record nel database simultaneamente:

1. Creazione dell'istanza `User` (con hashing della password).
2. Creazione dell'istanza `Profilo` collegata all'utente appena creato.

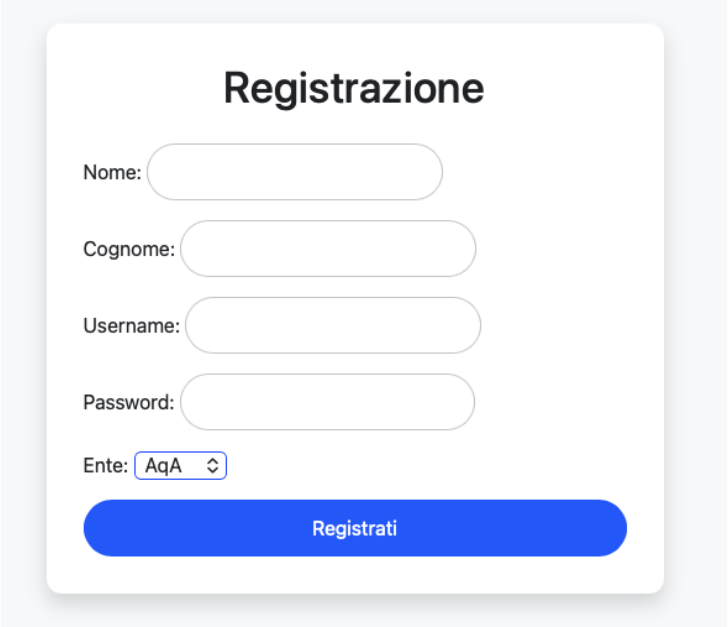

Figura 2: Registrazione.

3.3 Dashboard di Amministrazione

La dashboard riservata all'amministratore (*ADMIN*) è stata progettata per consentire una gestione granulare delle utenze. Per prevenire errori operativi critici, il sistema integra un controllo logico a livello di template che inibisce la possibilità per l'amministratore corrente di eliminare il proprio account (*Sessione Attiva*), garantendo così la continuità di accesso al pannello di controllo.

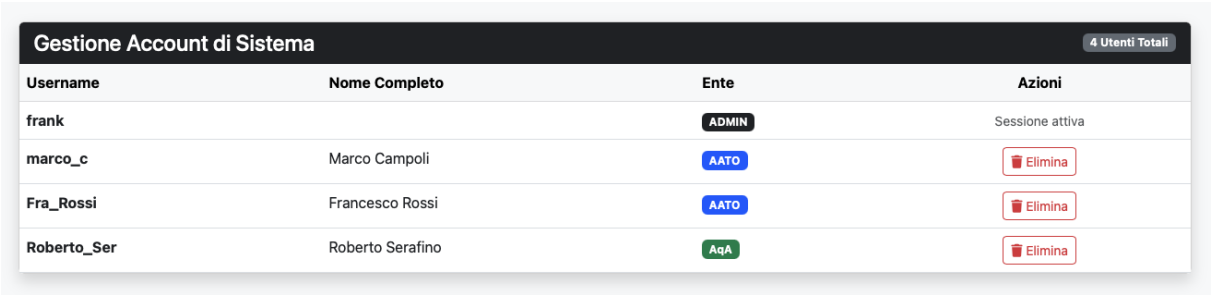

Figura 3: Schermata amministratore.

4 Interfaccia Utente (Frontend)

Il frontend è realizzato mediante il motore di templating di Django, integrato con **Bootstrap 5.3** per la responsività.

4.1 Struttura Base e Ereditarietà

Tutte le pagine ereditano da `base.html`, garantendo coerenza grafica. Il template base gestisce:

- **Header Fisso**: Include i pulsanti di navigazione contestuali (visibili solo se l'utente è autenticato) e il titolo del sistema.
- **Feedback Utente**: Un sistema di messaggistica (`messages`) mostra alert a scomparsa (es. "Credenziali non valide").
- **Loghi Istituzionali**: Una sezione dedicata ai loghi (Provincia di Mantova, AATO e AqA).

4.2 Dashboard Differenziate

Il sistema presenta due dashboard distinte, definite in `aato_home.html` e `tea_home.html`. Sebbene condividano lo stesso stile visivo (`main-box`), offrono funzionalità diverse tramite una griglia di pulsanti:

Funzionalità	**Dashboard AqA**	**Dashboard AATO**
Inserimento Dati	Nuovo Manufatto	-
Consultazione	Lista Manufatti	Lista Manufatti
Analisi	-	Ricerca Interventi
Cartografia	Visualizza Mappa	Visualizza Mappa
Documentale	Elenco Documenti	Elenco Documenti

Tabella 1: Matrice delle funzionalità per tipologia di utente

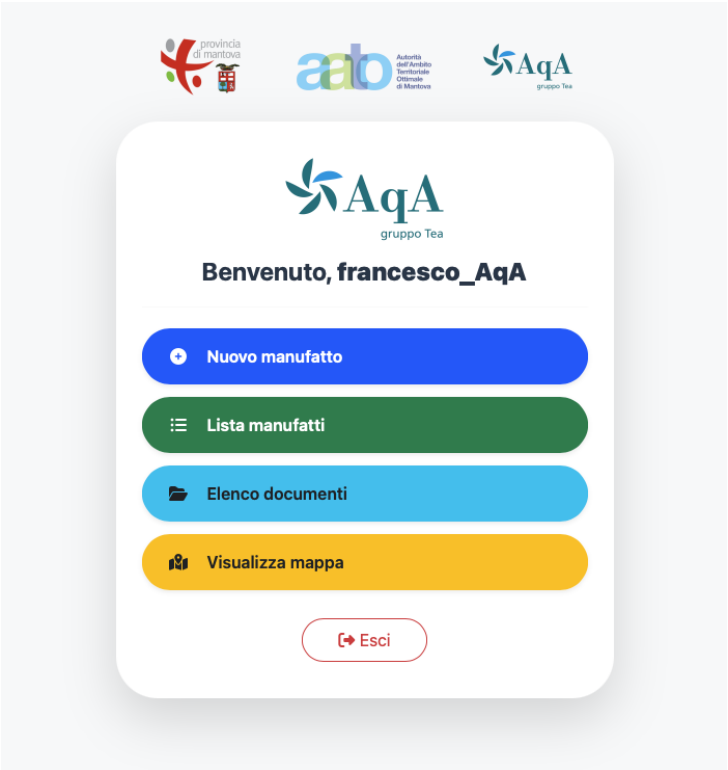

Figura 4: Schermata principale.

5 Sicurezza

L'implementazione rispetta i principi di sicurezza standard di Django:

- **CSRF Protection**: Tutti i form (Login e Registrazione) includono il token `{% csrf_token %}` per prevenire attacchi Cross-Site Request Forgery.
- **Password Hashing**: Le password non sono mai salvate in chiaro, ma gestite tramite le funzioni di hashing interne di Django durante la creazione dell'utente.
- **Controllo Input**: Il form `UserProfiloForm` valida l'univocità dello username prima della persistenza dei dati.

6 Conclusioni

Il modulo analizzato fornisce una solida base per il sistema di monitoraggio. La separazione logica tra `User` e `Profilo` permette una facile scalabilità qualora dovessero essere introdotti nuovi enti o ruoli. L'interfaccia utente, pulita e responsive, facilita l'operatività quotidiana degli operatori tecnici (AqA) e di controllo (AATO).

7 Architettura del Modulo Operativo e Gestione Dati

Il modulo operativo costituisce il nucleo del sistema, gestendo il ciclo di vita dei manufatti idraulici. L'architettura è stata progettata per gestire la complessità dei dati tecnici (idraulici e geografici) mantenendo al contempo un database normalizzato e performante.

7.1 Modellazione ER e Relazioni One-to-One

L'analisi del file `models.py` evidenzia una strategia di modellazione basata sulla separazione logica delle responsabilità. L'entità principale `Manufatto` funge da aggregatore, collegata tramite relazioni **One-to-One** a due tabelle satellite:

- **Info Idrauliche:** Questa entità incapsula la complessità tecnica, memorizzando parametri calcolati come gli Abitanti Equivalenti (ae_tot), le portate di progetto (Q_{nm}, Q_s) e i dati autorizzativi (Provincia/Consorzio). Questa separazione evita di appesantire la tabella principale con oltre 30 campi opzionali.
- **Info Geografiche:** Decoppia le coordinate spaziali (`latitudine`, `longitudine`) dall'anagrafica.

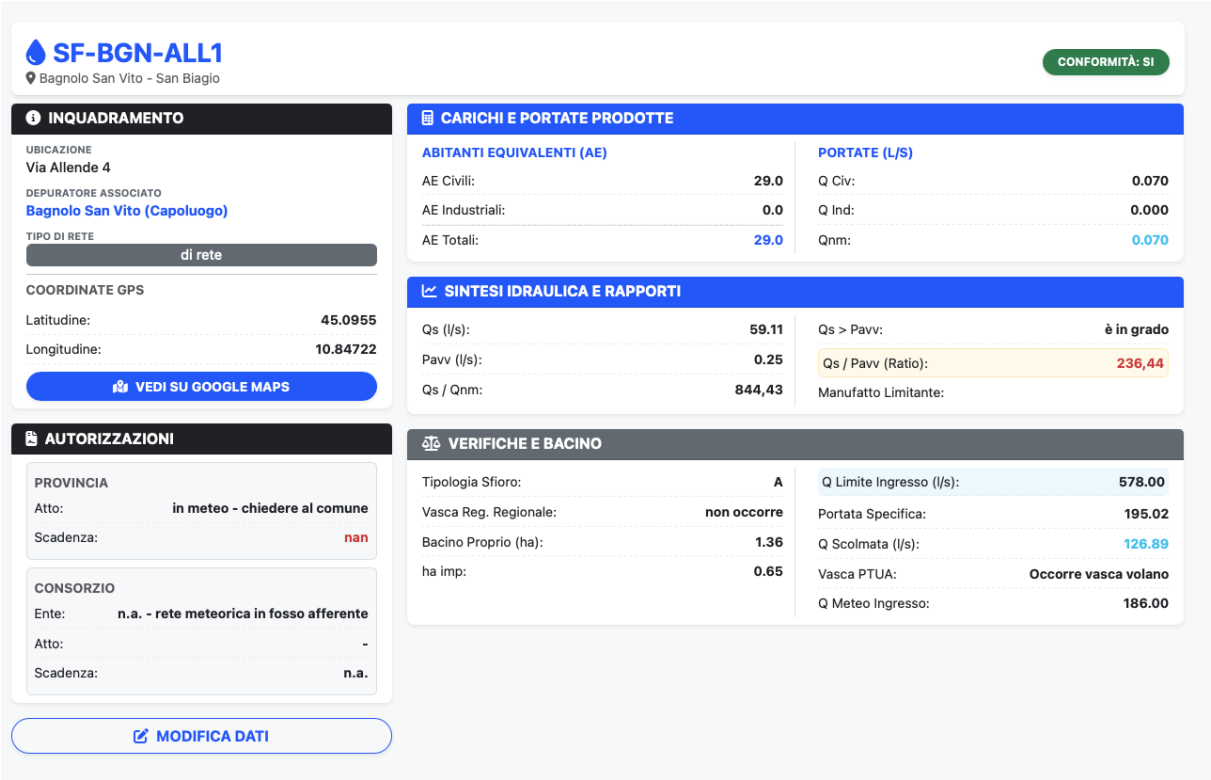

Figura 5: Schermata dati.

7.2 Gestione Transazionale dei Form

La vista `crea_manufatto` gestisce la persistenza atomica di un dataset eterogeneo, orchestrando in un'unica transazione le informazioni anagrafiche del manufatto, i parametri idraulici e le coordinate geografiche. Il flusso di salvataggio segue una sequenza rigorosa per garantire l'integrità referenziale:

1. Validazione simultanea dei tre form.
2. Persistenza dell'istanza padre `Manufatto` per ottenere la Primary Key (PK).
3. Creazione delle istanze figlie con `commit=False`.
4. Iniezione della foreign key del manufatto appena creato nelle istanze figlie.
5. Salvataggio definitivo delle relazioni.

8 Data Engineering e Interoperabilità (Excel)

Una delle componenti più critiche del sistema è il motore di importazione ed esportazione dati, basato sulla libreria **Pandas**, che permette la gestione massiva delle informazioni evitando l'inserimento manuale.

8.1 Algoritmo di Importazione (ETL)

La funzione `import_manufatti` in `views.py` implementa una pipeline ETL (*Extract, Transform, Load*) avanzata, progettata per essere resiliente a file Excel non standardizzati:

- **Header Detection Dinamico**: L'algoritmo non assume che l'intestazione sia alla riga 0. Effettua una scansione delle prime 100 righe alla ricerca della chiave semantica "codice". Una volta individuata, imposta dinamicamente quella riga come header del DataFrame.
- **Sanificazione dei Dati (Data Cleaning)**: Sono state implementate funzioni helper come `safe_float` e `clean_ratio`. Queste gestiscono le anomalie comuni nei file CSV/Excel italiani, come l'uso della virgola al posto del punto per i decimali, convertendo automaticamente stringhe come "nan", "None" o "-" in valori gestibili dal database.
- **Parsing delle Coordinate**: Una logica dedicata gestisce campi coordinate compositi (es. "45.123, 10.456"), effettuando lo split della stringa e il casting a float per popolare la tabella `info_geografiche`.

8.2 Esportazione e Denormalizzazione

La vista `export_manufatti` esegue l'operazione inversa. Aggrega i dati distribuiti sulle tre tabelle relazionali, denormalizzandoli in un'unica struttura piatta. Utilizzando l'engine `openpyxl`, genera uno stream binario MIME-type `application/vnd.openxmlformats...`, permettendo il download diretto di un report Excel formattato.

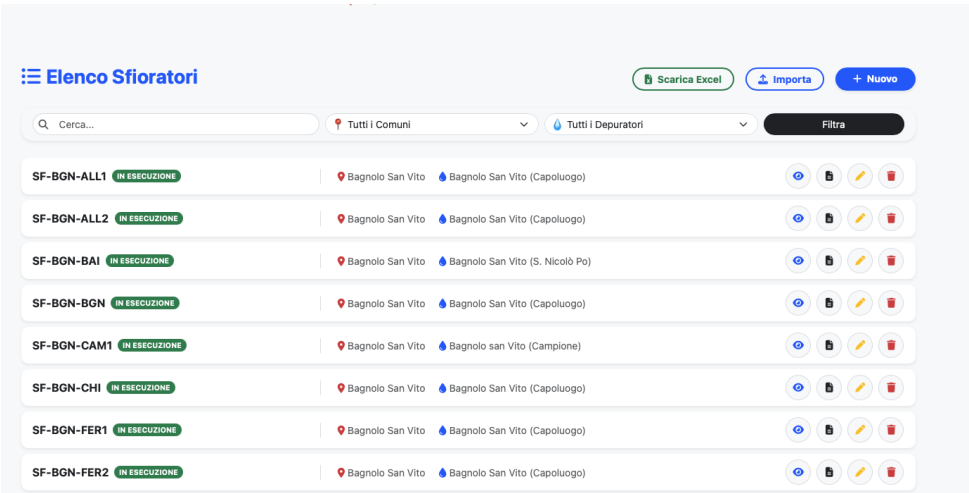

Figura 6: Lista.

9 Mappa e Ottimizzazione delle Query

La visualizzazione territoriale (`visualizza_mappa`) integra il framework JavaScript **Leaflet** con il backend Django.

9.1 Soluzione al problema N+1

Un aspetto tecnico rilevante è l'ottimizzazione delle performance database. Nella vista mappa, il recupero dei dati non avviene tramite iterazione standard, ma utilizza il metodo:

```
Manufatto.objects.all().select_related('info_geografiche')
```

Questo comando forza Django a eseguire una **SQL JOIN** immediata, recuperando manufatti e coordinate in un'unica query. Senza questa ottimizzazione, il sistema eseguirebbe una query aggiuntiva per ogni singolo marker sulla mappa (problema N+1), degradando le prestazioni all'aumentare dei punti.

Figura 7: Mappa.

10 Gestione Documentale Sicura

Il modulo documenti permette l'upload di file tecnici (PDF, immagini) associati ai manufatti.

- **Storage Partizionato**: I file non vengono salvati in un'unica cartella, ma distribuiti in un albero di directory basato sulla data (`/anno/mese/giorno/`). Ciò previene limiti del filesystem sul numero di file per directory.
- **Secure Download**: La funzione `scarica_documento` non espone mai l'URL diretto del file statico. Utilizza invece `FileResponse` per servire il file come stream. L'header `Content-Disposition` viene forzato a `attachment`, garantendo che il browser scarichi il file con il nome originale corretto, proteggendo al contempo la struttura delle directory del server.

11 Interfaccia Utente (Frontend Logic)

Le viste implementano logiche di filtraggio dinamico per migliorare la UX:

- **Filtri Contestuali**: I menu a tendina per filtrare per "Comune" o "Depuratore" nella `lista_manufatti` non sono statici. Vengono popolati tramite query `.values_list('...')` `.distinct()`, mostrando all'utente solo le opzioni per cui esistono effettivamente dati nel sistema, eliminando scelte vuote o ridondanti.
- **Feedback System**: L'uso intensivo del framework `messages` di Django fornisce feedback immediato (Success/Error) dopo operazioni critiche come l'importazione massiva o l'eliminazione di record.

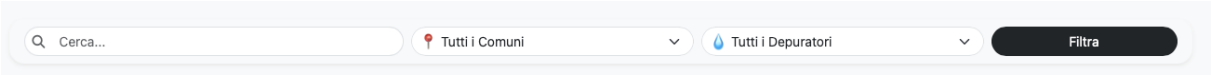

Figura 8: Filtraggio.

12 Ingegneria del Frontend e User Experience (UX)

L'interfaccia utente del sistema è stata sviluppata seguendo i principi del *Responsive Web Design*, utilizzando il framework **Bootstrap 5** integrato con il motore di templating di Django (DTL).

L'obiettivo primario è ridurre il carico cognitivo dell'operatore, trasformando dati tecnici grezzi in visualizzazioni intuitive.

12.1 Pattern di Visualizzazione Dati (Dashboard)

Il template `dettaglio_manufatto.html` rappresenta il punto nevralgico per la consultazione. Invece di una tabella piatta, è stata adottata una struttura a **Widget/Card** tematiche:

- **Gerarchia Visiva**: I dati sono raggruppati logicamente in "Inquadramento", "Carichi e Portate", "Sintesi Idraulica" e "Autorizzazioni".
- **Rendering Condizionale (Compliance)**: Il sistema valuta lo stato di conformità. Tramite i tag di template, viene generato un feedback visivo immediato:

```
<span class="badge {% if manufatto.info_idriche.e_conforme == 'SI' %}bg-success{% else %}bg-danger{% endif %}">
    CONFORMITA: {{ ... }}
</span>

```

  Questo permette all'ente di controllo (AATO) di individuare a colpo d'occhio le criticità (Badge Rosso) rispetto alle situazioni a norma (Badge Verde).

- **Formattazione Numerica**: L'uso del filtro `|floatformat:2` garantisce che i dati idraulici (spesso con molti decimali nel DB) vengano presentati con una precisione leggibile.

12.2 Gestione dell'Input Complesso (Accordion Pattern)

La creazione e modifica di un manufatto (`crea_manufatto.html`, `modifica_manufatto.html`) richiede la compilazione di oltre 40 campi. Per evitare una pagina eccessivamente lunga (scrolling fatigue), è stato implementato il pattern **Accordion**:

- **Segmentazione**: I tre form distinti (*Anagrafica, Idraulica, Geografica*) sono incapsulati in sezioni collassabili.
- **Interattività JavaScript**: Uno script dedicato (`toggleSection`) gestisce la visibilità dei div e la rotazione delle icone (`fa-chevron-down`), permettendo all'operatore di concentrarsi su una sezione alla volta.

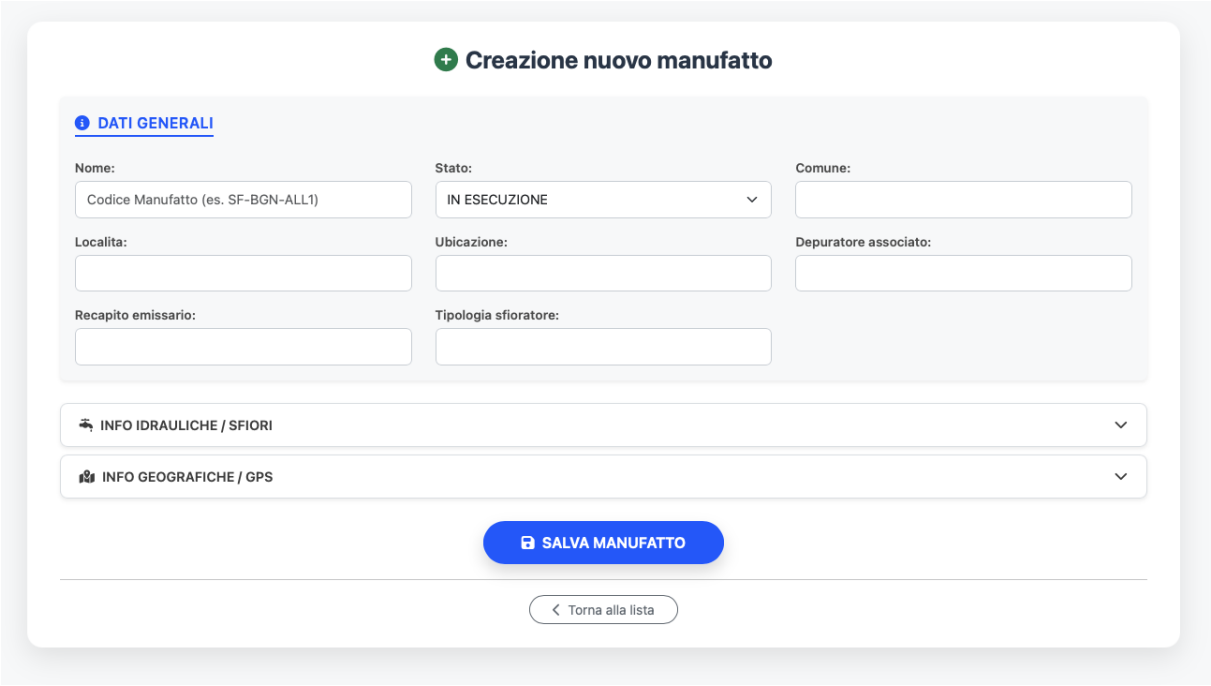

Figura 9: Schermata creazione manufatto.

12.3 Integrazione Mappa (Leaflet.js)

La cartografia interattiva in `visualizza_mappa.html` è realizzata client-side mediante la libreria open-source **Leaflet**.

12.3.1 Iniezione dei Dati (Data Injection)

Non vengono effettuate chiamate AJAX asincrone per popolare la mappa; per massimizzare la velocità, i dati vengono iniettati direttamente nel DOM durante il rendering server-side di Django. Il template itera sul queryset e genera codice JavaScript on-the-fly:

```
{% for m in manufatti %}
    var lat = parseFloat("{{ m.info_geografiche.latitudine|safe }}".replace(',',
     '.'));
    // ...
    L.marker([lat, lng]).addTo(map).bindPopup("...");
{% endfor %}
```

12.3.2 Gestione delle Anomalie

È stato implementato un meccanismo di *Sanitization* in JavaScript per gestire le differenze di localizzazione (virgola vs punto decimale) che spesso affliggono i dati importati da Excel, prevenendo crash della mappa in caso di coordinate malformate. Inoltre, la funzione `map.invalidateSize()` viene invocata per correggere problemi di rendering del canvas quando la mappa viene caricata all'interno di container dinamici.

12.4 Gestione Documentale e Upload

I template `gestione_documenti.html` e `upload_excel.html` curano l'interazione per il trasferimento file.

- **Feedback di Sicurezza**: Prima di operazioni distruttive (eliminazione manufatto o documento), vengono mostrati modali o alert di conferma (`conferma_elimina.html`) per prevenire cancellazioni accidentali.
- **Navigazione Contestuale**: I pulsanti "Torna Indietro" sono logici e dinamici, reindirizzando l'utente alla dashboard corretta (AqA o AATO) in base al gruppo di appartenenza rilevato nel context template.

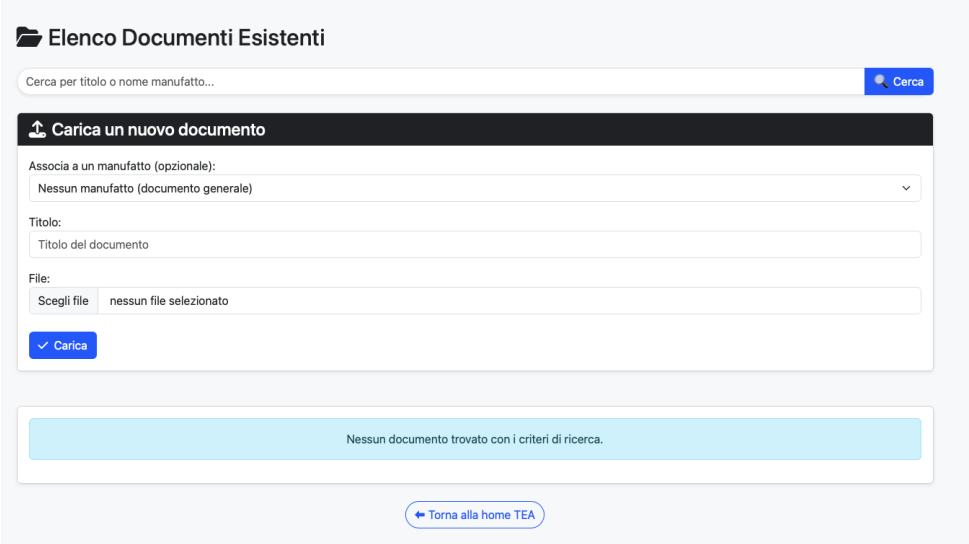

Figura 10: Pagina elenco documenti.

13 Conclusioni Generali

Il sistema "Monitoraggio Sfiori" rappresenta una soluzione *Full-Stack* completa. L'architettura backend garantisce la coerenza e la sicurezza dei dati idraulici complessi, mentre il frontend offre un'esperienza utente moderna e reattiva. L'integrazione profonda tra i modelli di Django, l'analisi dati con Pandas e la visualizzazione Mappa fornisce alla Provincia di Mantova uno strumento efficace per la gestione del servizio idrico integrato.